ENGINEERING ASSESSMENT & TARGET ARCHITECTURE · NEURO METRO

Porting AVUI to Godot

What moves across cleanly, what has to be rebuilt, what stops existing — and how the Godot project must be organised so artists and designers can work in it without touching code.

Subject	Scope	Target	Date
NEURO METRO: AVUI	**BATTLE SYSTEM + MAP**	**Godot 4.3+**	**31 August 2026**

Orientation — read this first

This document assumes no prior knowledge of the project. Everything it describes lives under `NEURO METRO/AVUI/`.

PATH	WHAT IT IS
`BATTLE SYSTEM/`	The combat app. `index.html` is markup only; behaviour in `src/*.js` (19 classic scripts, one global scope, load order = dependency graph); all styling in `styles/game.css`. **Read `ARCHITECTURE.md` for how it is built, and `README.md` for what the systems *are*** — the Line Manager, escape, AI profiles and the music decision are all documented there and nowhere else.
`MAP/`	Everything outside battles: the Barcelona L1–L6 network, routing, rides, objectives, the vault. Same conventions, its own `ARCHITECTURE.md` and a 1,262-line `README.md` covering guardians, lock-on enemies, rewards and the seam with battle.
`GDDs + Spreadsheets/`	`battle-system-config.xlsx` — **the source of truth for all content** — plus `AVUI_COMBAT_GDD.md` (§13 defines the map↔battle contract; **§16 lists fourteen traps already fallen into**), `AVUI_NAVIGATION_GDD.md`, `AVUI_PROGRESSION_GDD.md`.
`shared/config/csv/`	CSVs exported from that workbook. **Generated — never hand-edit.** A row added here survives only until the next export.
`shared/tools/`	Python. `export_csv.py` (workbook→CSV), `build_data.py` (CSV→`data.js`), plus validators.
`REBUILD.command`	Double-click after editing the spreadsheet. This is the entire designer loop and it must survive the port.

The two apps are deliberately walled off from each other and talk only through the §13 contract, today over an iframe and `postMessage`. **The wall is a working convention, not an accident — preserve it in the new structure.**

13,995	2,019	23	180	3	84
LINES JS + CSS	LINES PYTHON	SPREADSHEET SHEETS	TUNABLE RULES	IMAGE ASSETS TODAY	CSS KEYFRAME SETS

> **Verdict: the port is worth doing, and it has two distinct goals that must not be confused.** The first is *making it run* — that part is lopsided in a useful way: roughly 40% of the codebase (the whole simulation, the data pipeline, the rasterisers, the save system) moves almost mechanically, 40% is presentation that must be re-expressed rather than translated, and 20% is visual parity, which is an art re-tuning pass and the part most likely to be underestimated.
>
> The second goal is **making it authorable**, and it is the more important one. Today, adding an enemy is a spreadsheet row but changing how that enemy *looks* means editing JavaScript. In Godot it must mean opening a scene. Section 3

below is the binding architecture for that, and it should be treated as a requirement, not a suggestion — it is cheap to follow from the start and expensive to retrofit.

The single biggest structural win is that **the two apps become one project**: the iframe, the `postMessage` bridge, the opaque-origin autoplay problem and the whole `file://` compatibility layer are deleted rather than ported. The single biggest risk is that the look lives in stacked CSS filters and 75 hand-tuned keyframes; Godot reproduces all of it, but by different means and not by transcription.

2 · What exists today

LAYER	SIZE	HOW IT WORKS NOW
Simulation `model · kinds · rules · hooks · battle`	~1,190 ln	Pure logic, no DOM. Units, line slots, layer queue, 8 ability kinds (DAMAGE, SHIELD, CHARGE, SELFHARM, FEED, HEAL, ADDLAYER, DEBUFF), matchup lookup, an event bus, the turn loop and two enemy AI profiles.
Set pieces `boss · escape`	279 ln	The Line Manager's second wind and fall, and hold-to-flee. Both keyed off data (`units.role`, a rules value) rather than off an id — see 3.10.
Presentation (DOM) `view · fx · screens · dialogue · art`	~1,560 ln	Elements, classes, inline styles. 154 `classList` calls, 117 direct `style.` writes, 42 `getBoundingClientRect` reads driving animation geometry.
Presentation (canvas) `rings · gauge · mapview · font`	~1,270 ln	Software rasterisers. Pixels written into an `ImageData` at low resolution and upscaled with `image-rendering:pixelated`. No canvas strokes anywhere, deliberately.
Stylesheets `game.css · map.css`	2,400 ln	The entire visual identity. 419 custom-property references, 84 keyframe sets, 84 filters, 111 box-shadows, 190 transforms, 3 blend modes, 18 clip-paths, 84 `calc()`.
Map systems `journey · elements · travel · vault · player`	~2,500 ln	Routing, multi-leg runs, world state, weighted element rolls, the vault, persistence.
Audio `audio.js ×2`	~560 ln	A hand-built WebAudio graph: oscillators and noise buffers, exponential envelopes, a 12-step waveshaper bit-crusher, a feedback delay, a 7.2 kHz lowpass, and a clean bus for recorded music.
Data toolchain `Python ×7`	2,019 ln	Workbook → CSV → `data.js`. 180 tunable rules; nothing is a literal in `src/`.

CONTENT THE PORT HAS TO CARRY

12 enemies plus the player — of which **3 are bosses** (Fondo, Sant Antoni, and the Line Manager at Urquinaona) · 27 abilities in 8 kinds · 6 emotions · 9 status effects · 10 icon glyphs · 8 loadouts · 7 armor pieces · 6 travel elements · 4 objectives · 29 title moments · 16 line prompts · 207 dialogue persona rows · 10 world bands · 2 city statuses · 6 metro lines · 111 stations. These counts decide how many scenes exist, and they are all small enough that a scene-per-instance structure is comfortable rather than unwieldy.

> **RECENTLY ADDED — CHECK THESE ARE STILL CURRENT BEFORE ACTING**
>
> The build moves quickly. Since this assessment: `src/boss.js` (the Line Manager set pieces), `src/escape.js` (hold-to-flee), the `HEAL` ability kind, `ai_profile` as a real column, the Station Guardian warning banner, lock-on "hunter" enemies, and streamed rather than decoded boss music. All are covered below, but **re-read `BATTLE SYSTEM/README.md`, `MAP/README.md` and both `ARCHITECTURE.md` files before starting** — they are kept current and this document is a snapshot.

WORTH STATING UP FRONT

The architecture already separates simulation from presentation and keeps every tunable number out of code. That discipline is exactly what makes this port tractable — and it is also what makes the scene-based structure below achievable, because the data layer it needs already exists and is already authoritative.

3 · The target project — how it must be organised

The goal is a project a non-programmer can open, understand and change. Adding an enemy, restyling a status tag, giving one boss its own entrance animation or swapping a station symbol should all be things done by opening a scene and editing it — never by finding the right branch in a `.gd` file.

> THE GOVERNING RULE
>
> **One script per KIND. One scene per INSTANCE. Content in the spreadsheet. Look in the scene. Effects in shaders.**
>
> There is one `Enemy.gd`, not twelve. There are twelve enemy *scenes*, each one a visual document. An enemy's numbers come from its spreadsheet row; its appearance, animation and flourishes are nodes in its own scene. Nobody writes code to add either.

3.1 Project layout

```
res://
  data/                       generated from the workbook — never hand-edited
    emotions.csv  abilities.csv  units.csv  matchups.csv  status_effects.csv
    stations.csv  metro_lines.csv  travel_elements.csv  rules.csv  sounds.csv  …

  art/                        every image is a file an artist can replace
    emotions/    ANGER.png SURPRISE.png DISGUST.png JOY.png SADNESS.png FEAR.png
    icons/       BOLT.png DROP.png SPARK.png BURST.png ROT.png EYE.png
                 SHIELD.png CHARGE.png WARN.png GLASS.png
    stations/    ring.png ring_interchange.png dot.png dot_minor.png
    map/         line_segment.png line_corner.png marker_objective.png …
    items/  armor/  ui/panels/ (9-patch)  ui/tags/  fonts/

  shaders/                    visual effects live here, not in .gd
    glow.gdshader          replaces every CSS drop-shadow
    ramp.gdshader          the shared scrolling emotion gradient
    gauge.gdshader         the undulating MS/EC silhouette
    rings.gdshader         emotional layers; tier shape via f(angle)
    swim.gdshader          the turbulence displacement
    glitch.gdshader        the overload / crit wash
    pxr_corner.gdshader    the chamfered pixel corner

  components/                 reusable, nested everywhere
    Gauge.tscn            MS/EC bar — battle, ride HUD, profile card
    BarTag.tscn  StatusTag.tscn  Tooltip.tscn  FloatingNumber.tscn
    Station.tscn           one line slot: ring + icon + charge
    ChargeNode.tscn  TerminusArrow.tscn  CritSlot.tscn
    AbilityCard.tscn  LoadoutButton.tscn
    EmotionIcon.tscn       reads emotions.csv, shows the right art/

  battle/
    Battle.tscn            the fight shell — instances everything above
    enemies/
      _Enemy.tscn          THE BASE. Every enemy is an inherited scene.
      enemy_anger_strong.tscn   enemy_surprise.tscn   enemy_joy_weak.tscn  …
      _Boss.tscn           inherits _Enemy; adds the set pieces (3.10)
        boss_fondo.tscn  boss_sant_antoni.tscn  boss_line_manager.tscn
    arenas/  _Arena.tscn  arena_L1.tscn … per-line staging, boss backdrops
    setpieces/
      SecondWind.tscn      hush, roar, flash, the 5s climb, SECOND PHASE
      BossFall.tscn        two parting lines, shake + 7s sink
      EscapeRing.tscn      hold-to-flee ring closing under the finger
    fx/      HitFlash.tscn  LayerBreak.tscn  ColourWash.tscn  CritBeat.tscn
             SpeechBubble.tscn  WhiteFlash.tscn

  map/
    Map.tscn
    rides/   _Ride.tscn  ride_L1.tscn ride_L2.tscn … ride_L6.tscn
    markers/ ObjectiveMarker.tscn   the floating "?"
             StationMarker.tscn  CityStatusMarker.tscn
    warnings/
      GuardianBanner.tscn  3s platform warning; .manager variant
      AggroWarning.tscn    ⚠ + countdown ring on a lock-on enemy
    TripPreview.tscn       the route pop-up
    RewardPanel.tscn       queued, shown once the map settles to IDLE
    elements/ _TravelElement.tscn + one per travel_elements row
    EnemySprite.tscn       SHARED with battle — see 3.4

  scripts/                    one file per kind, and few of them
    DataTable.gd (autoload)  Enemy.gd  Boss.gd  Ride.gd  Gauge.gd  StatusTag.gd
    Station.gd  TravelElement.gd  Battle.gd  Escape.gd
    Hooks.gd  Kinds.gd  Model.gd  Rules.gd
```

3.2 Inherited scenes, not copies

The instinct — "duplicate a scene, rename it, edit it" — is right, and Godot gives you something strictly better for it. **Scene → New Inherited Scene** creates a child that keeps its link to the base: every node, script and default is carried over, anything overridden in the child stays overridden, and *a later change to the base propagates into every enemy that inherits it*. Plain duplication gives you twelve unconnected copies and a fix that has to be applied twelve times.

> THE WORKING RULE
>
> **Inherit by default.** New enemy = New Inherited Scene from `_Enemy.tscn`, save as the unit's id, set the exported fields, drop in whatever art or animation it needs. **Duplicate only for a deliberate fork** — something that genuinely is not that kind of thing any more. Both work; the difference is whether future fixes reach it.

3.3 What lives in the Inspector, and what lives in the sheet

This split is the one thing most likely to go wrong, and it goes wrong the same way every time: a number ends up in two places and they drift. The rule follows directly from the project's existing law that the spreadsheet is the source of truth.

	LIVES IN THE SPREADSHEET	LIVES IN THE SCENE / INSPECTOR
What it is	Rules and balance — anything that changes how the game *plays*.	Presentation — anything that changes how it *looks, sounds or moves*.
Enemy example	`max_ms`, `layers`, `pool`, `ai_profile`, `spawn_lines`, `drops`, `tier`	Its sprite, its idle animation, its particle nodes, its entrance timing, the shader params on its rings, an extra light, a bespoke flourish
Read by	`DataTable` autoload, at runtime, keyed by id	The scene file itself; nothing has to read it
Edited by	Designer, in Google Sheets / Excel, then `REBUILD.command`	Artist, in the Godot editor
Never	Never holds a texture path or an animation name	**Never holds a balance number.** If a value decides an outcome, it is a row.

The awkward middle — columns like `scale`, `bg_bright`, `fx`, `theme_loop` — already exists in the `units` sheet and is presentation expressed as data, because the browser build had nowhere else to put it. **Most of it should move into the enemy's scene and the columns retired.** `fx=PAPARAZZI` is today a string the code must branch on; as a scene it is simply a node the boss has and the others do not, and no code knows the word.

3.4 One script, many instances, configured in the Inspector

Every enemy runs the same `Enemy.gd`. The script exposes presentation fields to the Inspector and pulls its rules from the table by id. Two things make it work for a non-programmer: `@tool`, so the values resolve live in the editor, and the convention that **the scene's filename and root node name are the spreadsheet id**.

```
@tool
class_name Enemy extends Node2D

# The link to the sheet. Defaults to the node's own name, so an inherited
# scene saved as `boss_fondo.tscn` is wired up by the act of naming it.
@export var unit_id: StringName = &""

# --- Presentation. Artists own everything below. ---
@export_group("Look")
@export var sprite:      Texture2D
@export var idle_anim:   StringName = &"float"
@export var ring_shape:  ShaderMaterial          # tier silhouette
@export var flourish:    PackedScene             # optional, e.g. PAPARAZZI
@export_range(0.5, 3.0) var backdrop_brightness := 1.0

# --- Rules. Read-only mirror of the sheet, shown so you can SEE them ---
#     in the Inspector without them being stored in the .tscn.
@export_group("From the spreadsheet (read-only)")
@export_storage var _row: Dictionary

func _ready() -> void:
    if unit_id == &"": unit_id = name          # name IS the id
    _row = DataTable.units.get(unit_id, {})
    if _row.is_empty():
        push_warning("No units row for '%s' — check the sheet or rename the scene." % unit_id)
```

Three consequences worth naming:

- **The sheet's numbers are never written into the `.tscn`.** They are read at load and displayed; the scene stores only presentation. There is exactly one place a balance number exists, which is the property the current project already protects.
- **Renaming the scene rewires it.** No id typed twice, nothing to keep in sync by hand.
- **A mismatch is loud.** A scene whose name matches no row warns in the editor. The reverse check — a row with no scene — belongs in the same validator that `build_workbook.py` already uses to refuse dangling references; extend it into an `EditorPlugin` so both directions are caught before anyone plays.

The same pattern covers everything else: `Ride.gd` across `ride_L1…L6.tscn` keyed by line id, `TravelElement.gd` across the six element scenes, `StatusTag.gd` across the nine statuses, `Station.gd` across the 111 stations (which stay data-driven instances of one scene rather than 111 files).

3.5 Everything visible is a component scene

Anything that appears in more than one place, or that anyone might want to restyle, is its own scene and is instanced — never rebuilt a second time. That is what makes a change propagate instead of being applied three times and missed once.

COMPONENT	APPEARS IN	WHY IT MUST BE A SCENE
Gauge.tscn — the MS/EC bar	Battle (player + enemy), the ride HUD, the profile card	Drawn three times today. One scene means the bar is restyled once and every screen follows.
EnemySprite.tscn	The ride (an approaching entity) and the fight (the opponent)	The same creature in two contexts. Authored once, instanced in both — and each enemy scene supplies its own art to it.
Station.tscn — a line slot	Both fighters' Metro Lines, the build preview	Ring + emotion icon + charge segments. Composited from `art/`, so changing the ring changes every slot in the game.
StatusTag.tscn / BarTag.tscn	Both units' status rows, the bars, the ride HUD	The stylesheet's "one shape per role" rule becomes structural instead of a convention people have to remember.

CritSlot.tscn	The crit beat, then the next line built	It flies from the tag into the line and later appears as a real slot. One scene, two moments, no ghost duplicate.
ObjectiveMarker.tscn — the floating "?"	Over any station with something in it	Animation and colour-coding by emotion in one place, instanced 4+ times.
TripPreview.tscn	The map, on route selection	A self-contained pop-up an artist can open, restyle and animate without entering the map's code.
Tooltip.tscn	Abilities (long press) and statuses (tap), in both apps	Already deliberately identical across the two apps — a shared scene makes that true by construction rather than by copying.
GuardianBanner.tscn	The platform, before a Guardian or a Line Manager fight	Three seconds of warning, colour-coded to the enemy's emotion, with a `.manager` variant that shakes harder and says different words. One scene, two states, chosen from `units.role`.
AggroWarning.tscn	Over any lock-on enemy drifting past on a ride	The ⚠ sign and the countdown ring — *this one is coming* and *how long you have*. Currently drawn procedurally so it holds proportions from a WEAK dart to a STRONG spined shape; as a scene, that becomes a scale on a sprite.
SpeechBubble.tscn	Every enemy line: INTRO, WINNING, LOSING, DEFEAT, and a boss's PHASE2 / DEFEAT2	207 persona rows feed one bubble. Restyling dialogue is one scene, and the boss's extra states need no second implementation.
RewardPanel.tscn	After a cleared objective — at most three times in a save	Queued during the wipe home and flushed once the map is IDLE. Its rarity is the design; keeping it a scene keeps that moment editable without touching the map's flow.

3.6 Shaders instead of draw code

Every effect currently produced by CSS filters or by per-pixel JavaScript should become a `.gdshader` with named uniforms. A shader is a file, it hot-reloads, its parameters appear in the Inspector as sliders, and it runs on the GPU. That is better on every axis that matters here — including that someone who is not a programmer can open one, move a number and see the result immediately.

TODAY	BECOMES
Stacked `filter: drop-shadow()` — 68 declarations	`glow.gdshader` with `colour`, `inner_radius`, `outer_radius`. One material, reused; exposed per node.
`gauge.js` — 228 lines writing the undulating bar pixel by pixel	`gauge.gdshader` on a TextureRect inside `Gauge.tscn`. The half-period sine silhouette, the scrolling emotion ring, the ceiling chevron and the overcharge spill are all fragment maths.
`rings.js` — concentric layers, tier silhouettes via `hypot(dx,dy)/f(angle)`	`rings.gdshader`. The shape function is one line of GLSL; each enemy scene assigns its own material, so a new silhouette is a shader param, not a code branch.
`--ramp` + `background-attachment:fixed` — the shared gradient sheet	`ramp.gdshader` sampling `SCREEN_UV` with a scroll uniform. Simpler and more correct than the CSS hack it replaces.
`feTurbulence` + `feDisplacementMap` — the swim	`swim.gdshader` with a `NoiseTexture2D`. ~20 lines, cheaper than the SVG filter.
The `hue-rotate/invert/saturate` glitch keyframes	`glitch.gdshader` as a full-screen pass, driven by an `AnimationPlayer` writing uniforms — a timeline you can scrub.
`.pxr` — a 20-point clip-path on every panel	A 9-patch `StyleBoxTexture` in the Theme (preferred: it makes the corner an asset), with `pxr_corner.gdshader` as the fallback for non-panel cases.

THE ONE EXCEPTION

Keep *geometry* in code where it is genuinely geometry: the map's routing, which station a link connects to, where a label can fit. A shader draws; it does not decide layout. The rule is **shaders for appearance, nodes for structure, scripts for rules**.

3.7 Art becomes files

Today, station rings, charge nodes, arrows and all ten icon glyphs are built at runtime from 8×8 string grids in `src/util.js` and `src/art.js`, and the map is rasterised pixel by pixel. **Every one of the static ones should be exported once to PNG** (or authored as SVG and imported), so an artist can open, revise or replace them without reading code.

Export them *from the existing generators* — the grids are already the artwork, so a short script writes the whole set at the correct size and nothing is redrawn by hand. After that the string tables are deleted.

ELEMENT	TODAY	IN GODOT
6 emotion symbols 10 ability/status icons	8×8 grids in `ICONS`	**PNG per symbol** in `art/emotions/`, `art/icons/`. Named by id so `emotions.icon` resolves by filename with no lookup table — which also fixes the recorded bug where a sheet asked for `BURST`, no table had it, and Surprise silently wore Anger's symbol.
Station rings, charge nodes, terminus arrows	`stationSVG()`, `chargeSVG()`, `arrowSVG()`	**PNG**, composited inside `Station.tscn` — ring sprite plus icon child.
Map lines and station dots	Rasterised per pixel, redrawn at every zoom	**PNG** — dots and interchange markers as sprites; links as textured `Line2D`. See 3.8.
Item and armor symbols	Do not exist yet	**PNG** from the start, keyed by row id.
Panels, chips, tags	CSS clip-path + box-shadow + gradient	**9-patch PNG** in a Theme. One texture set, ~15–25 pieces.
UI type	Whatever monospace the device has, thickened with a text stroke	**A shipped font file** — mandatory, Godot has no system font stack.
The MS/EC bar, the layer rings	Per-pixel canvas	**Stay generated — as shaders.** These animate and are driven by live values; a PNG cannot undulate or breathe. Shader, inside a component scene.

WHERE THE LINE FALLS

Static → file. Animated or value-driven → shader. Never a hand-drawn string grid in a script. A symbol that always looks the same is a PNG an artist owns. A bar whose silhouette responds to two live numbers is a shader. Neither one is code an artist has to read.

3.8 The map as a scene, not a picture

The map is currently one canvas rasterised by hand at every zoom, for a stated reason: canvas strokes and `fillText` antialias regardless of `imageSmoothingEnabled`, so drawing normally would give soft-edged mush when upscaled. **That reason does not apply in Godot.** A low-resolution `SubViewport` upscaled with nearest-neighbour gives crisp pixels for free.

Which unlocks the version an artist can actually work in: **stations and lines become nodes inside that viewport** — `StationMarker.tscn` instanced per row, links as textured `Line2D`, labels in the real bitmap font, markers and city-status effects as their own scenes. The map becomes something you open and look at, with a selectable node per station, instead of 975 lines of rasteriser. Camera and zoom stay code; the picture stops being code.

The `squareLink()` rule (every link snapped to 0°/45°/90°) and the label-placement policy stay as scripts — they are structure, and hand-maintaining 111 exact angles is what they exist to avoid.

3.9 Naming, so the whole thing stays navigable

- **Scene filename = root node name = spreadsheet id.** `boss_fondo.tscn` ↔ root `boss_fondo` ↔ `units.id`. This is what makes sync-by-name work and what lets a validator check both directions.
- **Base scenes are prefixed with an underscore** — `_Enemy.tscn`, `_Ride.tscn`, `_TravelElement.tscn` — so they sort to the top and read as "do not instance me directly".
- **Art files are named by the id they serve**, not by what they depict: `art/icons/BOLT.png`, not `lightning.png`. Resolution by filename means no mapping table can go stale.
- **One script per kind, named for the kind** — and if a second script appears next to a scene, that is the signal that something belonged in the base or in a shader.

3.10 Set pieces are scenes, not sequences buried in code

The most orchestrated things in the game are the boss and guardian moments, and they are the strongest argument for this whole structure — because **the current codebase already works this way, and says so.** `src/boss.js` opens by stating that it is keyed off `units.role = LINE_MANAGER`, not off an id, so that all six planned Line Managers inherit the entire sequence from one cell each and nothing in the file knows which one. That is "one script per kind, one instance per scene" already, expressed in the only vocabulary a browser gave it. Godot lets it be expressed properly.

WHAT A LINE MANAGER DOES THAT NOTHING ELSE DOES

SET PIECE	WHAT HAPPENS NOW	AS A SCENE
The second wind `bossPhaseGate()`	She cannot be killed below `bossPhaseAt` (20%). Music stops over `bossHushMs` (520 ms); the interface is hushed to arena + player layers only; `boss_roar`; she speaks her `PHASE2` line; the screen white-flashes `bossFlashes` (4) times; then her stamina climbs to `bossPhaseTo` (50%) over `bossRechargeMs` (5 s) that you are made to sit and watch; one more flash; a different song; `SECOND PHASE`.	**SecondWind.tscn** with one `AnimationPlayer` track. Every beat above is a keyframe on a timeline you can scrub, re-time and preview — instead of ~70 lines of interleaved `await`, `sleep` and class toggles.
The fall `lineManagerFall()`	No particle burst and no clash wave. Music stops, she gets *two* parting lines (`DEFEAT` then `DEFEAT2` — states only a Line Manager reaches), then shakes her way off the bottom of the screen over `bossSinkMs` (7 s). "A thing that was the embodiment of an emotion does not pop."	**BossFall.tscn**. Note the existing design: the shake is a separate, faster layer *under* the slow sink, because one keyframe list doing both would have to spell out every jitter at every depth. In Godot that is simply two animation tracks, which is what it always wanted to be.

THREE RULINGS INSIDE THESE THAT MUST SURVIVE THE PORT

The gate intercepts the killing blow. "At 20%" cannot mean "when the bar lands on 20%" — a heavy hit goes from 34% to below zero and never passes through it, and she would die with her set piece unplayed. The test is `ms <= floor`, which includes dead, and the first thing the gate does is put her back on the line.

She keeps her charge. Charge is measured against the MS ceiling, so restoring her to 50% left what she was holding sitting above it — and the boss who had just declared herself the embodiment of an emotion came back *overloaded*, feeding the player and hurting herself. The rule is right; that is the wrong moment for it.

The recharge runs on the display clock, not the 12 fps one. Five seconds of a bar climbing is direct manipulation of a value, like the map's camera tweens — not an animation of game state. It is a deliberate exception to the house cadence, and porting it onto the 12 fps tick would make it stutter.

WARNINGS — THE UI THAT SAYS SOMETHING IS COMING

MOMENT	WHAT THE PLAYER GETS	SCENE

Station Guardian the fight you cannot refuse	`guardianWarnMs` (3 s) of banner, then it starts on its own. **No button** — there is no choice, and offering one would be a lie about what a Guardian is. A ⚠ in the enemy's emotion colour, `GUARDIAN ENTITY MANIFESTING`, a depleting rule, `map_flash`, and the platform thrown by a screen shake at 0.8×.	**GuardianBanner.tscn** — one scene, a `manager` flag chosen from `units.role`. Text, colour, sound and shake amplitude are all exported fields, so retuning the arrival is an Inspector job.
Line Manager the same door, different words	`LINE MANAGER HAS APPEARED`, `map_screech`, and the shake at 1.4× — harder, because she is the harder thing. "A Guardian *manifests*, which is a thing arriving; she *appears*, which is a thing that was always there deciding to be seen."	
Lock-on enemies on the ride	Some entities fix on you and count down; when it runs out the fight happens whether you wanted it or not. A countdown ring `aggroRingThick` (2 px) deep drawn as concentric passes, and a ⚠ in the enemy's own colour flashing at `aggroWarnHz` (4.5). The ring says *how long*; the sign says *this one is coming*.	**AggroWarning.tscn**, instanced onto the entity. `aggroChance` (0.25) is a property of the spawn, and `aggroTiers` gates it to WEAK|REGULAR — a hard fight you did not choose is a punishment.
A "?" that is a fight	`objectives.card_tag` decides what the station card admits: `ENTITY` shows ⚠ EXTREME EMOTIONAL DISTURBANCE, blinking, in the objective's colour; `TREASURE` shows OPPORTUNITY FOR TREASURE. Blank falls back on whether the row names a `unit`.	A state on the station card scene. The point is that the marker alone must not be the only warning — walking into a Line Manager expecting a chest reads as the game not having told you.
Escape the one that is never offered	Hold anywhere on the arena and a ring closes under your finger over `fleeHoldMs` (3 s). It costs `fleeSegmentCost` (5) Track Segments — the units the ride was spending. Never a button: "a control that says *leave* every turn is an interface arguing against its own subject."	**EscapeRing.tscn**. It must refuse on the controls, during the opening, mid-resolution, **during a Line Manager's second wind**, and once the fight is over — and re-check at the moment it fires, because three seconds is long enough for the fight to have moved on.

> **WHY THIS SECTION IS THE PROOF, NOT THE EXCEPTION**
>
> Every one of these is currently reached through a data column — `units.role`, `objectives.card_tag`, `ai_profile`, `aggroTiers` — with the behaviour written once and inherited. The project already refuses to special-case by id. The Godot structure in this section is not a new discipline being imposed on the codebase; it is the discipline the codebase already has, finally given somewhere to live that an artist can open.

4 · Easy to port

NEAR-MECHANICAL · SOME OF IT IMPROVES

Either pure logic or pure arithmetic. GDScript is dynamically typed with dictionaries and arrays, so the shape of the code survives; in several cases Godot's built-in equivalent is better than what had to be hand-rolled for the browser.

The whole simulation core `model · kinds · rules · hooks · battle`

Touches no DOM by design. Units, the fixed-length line array, the layer queue, overload derivation, status readers, criticals, the matchup lookup and the enemy AI are plain data transformations.

> **IN GODOT** Plain `RefCounted` classes with `class_name`, in `scripts/`. The `Kinds.define()` registry stays a dictionary of callables — it is already the right shape, and it is the extension point most new mechanics land on. Keep the `Hooks` bus rather than converting to signals: `emit` currently awaits each listener so a listener may animate, and Godot signals do not.

The spreadsheet pipeline `shared/tools/*.py`

The Python half needs no changes. Only the emit target moves: instead of wrapping CSV bodies in JS template literals, write the CSVs straight into `res://data/`.

> **IN GODOT** Godot reads CSV natively with `FileAccess.get_csv_line()` — the entire reason `data.js` exists (a browser cannot read a file from disk) disappears. A `DataTable` autoload replaces the generated `const` block.
> **TWO DETAILS A LOADER MUST HONOUR:** several sheets carry a comment row above the header, and every sheet carries a `LIVE` / `planned` marker row *beneath* it — both must be skipped. And `SCHEMA.list` columns are keyed by column NAME across all sheets, so a column made a list in one place becomes a list everywhere; read them tolerantly.

Saves and persistence `vault.js · player.js · firstrun.js`

17 `localStorage` call sites, JSON-serialised, with a v1→v2 migration path already written.

> **IN GODOT** `FileAccess` against `user://`. Strictly better: no quota, no silent clearing by the browser, no try/catch around every read. The migration logic ports unchanged.

The 12 fps house clock `rings.js tick() · STEP_MS = 83`

The most load-bearing convention in the project: one heartbeat, and every CSS animation manually forced onto it with `steps(duration/83ms)`.

> **IN GODOT** An accumulator in `_process`, or a fixed tick for the animation layer. The awkward part — that CSS animations run on their own clock and drift unless quantised by hand — is gone, because everything genuinely shares one tick. The map's deliberate second clock for camera motion is still expressible.

The audio bus topology `audio.js initAudio()`

`sfxBus → master → crusher → lowpass → out`, with `musicBus → cleanBus → out` beside it and a feedback delay.

> **IN GODOT** Buses in the mixer, effects as resources — and the whole chain is built in, including the crusher: `AudioEffectDistortion` in **LOFI MODE** is a bit-crusher, so `crusherSteps` maps onto it directly. `AudioEffectFilter` gives the 7.2 kHz lowpass and `AudioEffectDelay` has feedback. Declarative, visible in the editor, and the documented bug where one hidden gain silently attenuated the music 9 dB below the sheet's number becomes structurally hard to write.

The gapless theme handoff `theme-opening.wav → theme-loop.wav`

Currently requires decoding both to PCM up front and scheduling the loop on the audio clock at exactly `t0 + opening.duration`, with a non-gapless fallback for `file://`.

> **IN GODOT** `AudioStreamPlaylist`. One resource; the fallback path is deleted. Per-enemy theme swaps (`theme_opening` / `theme_loop` on the units sheet) become an exported field on the enemy scene.

The map↔battle seam `battle-bridge.js · handoff.js`

Today: build an `encounterDescriptor`, open an iframe, negotiate over `postMessage` across an opaque origin, delegate autoplay permission, hand a result back.

> **IN GODOT** A function call taking a Dictionary or a custom `Resource`. The §13 contract stays exactly as written — it was designed as an interface and it survives. The transport around it need not exist.

One-shot animations: shakes, floating numbers, flyers, tags `fx.js`

Currently `Element.animate()` with keyframe arrays, awaited via a helper.

> **IN GODOT** `Tween` is a direct and better replacement, including `await tween.finished`. Better still, each of these becomes a small scene (`FloatingNumber.tscn`, `CritSlot.tscn`) with its own `AnimationPlayer`, so the motion is editable on a timeline rather than in a keyframe array.

Tooltip markup `tipMarkup() · abilities.blurb`

`*emphasis*` and `{TOKEN}` resolved against a keyword colour map, unknown tokens degrading to plain ink.

> **IN GODOT** `RichTextLabel` with BBCode, inside `Tooltip.tscn`. The resolver becomes a string rewrite into `[color=…]`; graceful degradation is preserved for free.

Enemy AI profiles `ai_profile — GREEDY_MAX_DAMAGE · DEBUFF_FIRST`

Pure scoring logic over the ability pool, with the documented catch that a DEBUFF scores zero on damage-per-slot and needs its own branch, and that an attack which also hangs a status scores ×4 for the debuff profile.

> **IN GODOT** Straight port. Worth keeping as a `Callable` per profile in one dictionary, so adding a third brain is a row plus one function — the same shape as `Kinds`.

Boss and guardian sequencing `boss.js · GuardianBanner · encounter.js`

Long awaited sequences: hush, speak, flash, a watched 5-second climb, a 7-second sink, a 3-second platform warning — currently ~280 lines of interleaved `await`, `sleep` and class toggling.

> **IN GODOT** These become `AnimationPlayer` timelines inside the set-piece scenes of 3.10, awaited with `await anim.animation_finished`. This is one of the clearest wins in the port — the sequence stops being code and becomes something you can scrub, and the boss's timing is retunable without a programmer. Keep the display-clock exception for the recharge.

5 · Tricky but doable

REAL WORK · KNOWN SOLUTIONS · SOLVE ONCE, REUSE

Each has a clear answer, but the answer is a different technique, so it must be designed once and then applied everywhere. Most of the port's hours live here — and most of these become *assets or shaders someone else can edit*, which is the point.

Glows — `filter: drop-shadow()` everywhere `68 declarations`

Stacked coloured drop-shadows are how nearly everything reads as lit: gauges, active stations, the terminus arrow, shields, crit slots, the logo, map labels. Godot has no per-node drop-shadow.

> **RECOMMEND** One `glow.gdshader` exposing colour and two radii — matching the existing two-shadow pattern — applied as a material and set per node in the Inspector. Optionally a `WorldEnvironment` with 2D glow at low strength underneath (needs HDR 2D on the viewport). Budget this as the largest single line item; it touches almost every scene.

Layout — flexbox, `calc()`, `vw` / `dvh` `138 flex rules · 81 calc()`

Godot's containers cover most of what flex does here, but not `flex: 1` semantics exactly and not viewport-relative arithmetic.

> **IN GODOT** Containers plus anchors, with the fixed 420×920 portrait frame becoming the project viewport at `stretch_mode = canvas_items`, `aspect = keep`. Every `calc()` becomes a theme constant or a line of code. Mechanical, but there are 81 and each is a small decision.

Synthesised sound effects `the sounds sheet — no audio files exist`

Every SFX is generated at runtime from a row: wave type, start and end frequency, duration, gain, echo. Godot has no per-voice synthesis graph.

> **RECOMMEND** Pre-render at build time — extend the Python toolchain to render each `sounds` row to a `.wav` with numpy. The spreadsheet stays the source of truth, the designer loop is unchanged, runtime cost drops to zero, and the result is *a folder of audio files an artist can replace by hand* — which the current architecture makes impossible. The bus chain (crush, lowpass, delay) stays live, so the character is unchanged.

Rebuilding the map as nodes `mapview.js — 975 lines, 111 stations`

The rasteriser ports as code easily, but GDScript's per-pixel loops are far slower than JIT-compiled JavaScript, and this redraws the whole buffer whenever the camera moves.

> **RECOMMEND** Do not port it — replace it, per 3.8. Sprites and `Line2D` inside a low-res `SubViewport`. This solves the performance problem and the authorability problem in the same move, and is the biggest available simplification in the whole port. Keep the rasteriser only where output cannot be expressed as nodes.

Position-driven effects `flyStrike · zoneBox · 32 rect reads`

Flyers launch from a measured lane centre and land on a measured target rect; floating numbers are placed by walking `offsetParent`, because the enemy sprite is permanently mid-transform and its client rect lies.

> **IN GODOT** `global_position` and `get_global_rect()`, which are always true and never depend on a transition having settled. Better still, targets become named `Marker2D` nodes inside the relevant scenes, so where a hit flies to is something an artist can drag.

Gestures: long-press vs swipe vs tap `wirePanelGestures`

The interface has no button bar: tapping a station removes it, tapping the terminus arrow departs, tapping elsewhere on the line opens Emotions, a long press opens a tooltip — with the press cancelled by drag and the following click swallowed.

> **IN GODOT** `_gui_input` with hand-written state, plus `mouse_filter` to recover the specificity ordering CSS event bubbling gives today. No built-in gesture recogniser; the arbitration logic ports as logic, the plumbing is new.

Music: streamed or decoded, and why it matters `audio.js · the boss tracks`

A hard-won memory decision. `decodeAudioData` holds a track as Float32 PCM at ~337 KB *per second* — decoding all four tracks would allocate 134 MB, and 105 MB of that is the two Line Manager tracks. Decoding buys exactly one thing: the sample-accurate join between the theme's opening and its loop. A boss track handed over as one file has no such join, so it streams through an `<audio loop>` element instead.

> **IN GODOT** The distinction largely dissolves — Ogg/MP3 streams are played without being fully decoded to PCM, so the memory cliff goes away and the gapless join is a playlist resource. **BUT CARRY THE MASTERING NOTE ACROSS:** element volume is capped at 0–1 today, so the streamed tracks were mastered to where the decoded theme *ends up* after `musicVolume` (1.6). Once every track goes through the same bus, that compensation is wrong and levels must be re-measured, or the boss music arrives quiet.

Brightness that is not `alpha × 3` `bg_bright — the Line Manager is 3`

Alpha saturates: past about 1.8× the picture stops changing, so asking for three would quietly get you two. The current code spends `bg_bright` on three things at once — tint opacity, then lifting the colour toward white, then pushing the falloff further down the panel so the light fills the room instead of hanging in the top corner.

> **IN GODOT** Natural as a shader with three uniforms, or as a light with real HDR values where alpha compositing has no ceiling. Preserve the property that below ~1.7× every ordinary enemy renders identically to before — that is what keeps the boss special rather than making everything brighter.

Type `ui-monospace · font-weight:900 · -webkit-text-stroke`

The game borrows whatever monospace face the device provides and thickens it with a text stroke, precisely because mobile fallbacks ship no bold.

> **IN GODOT** Ship a licensed font file; `FontVariation` for weight, `outline_size` for the stroke. This also makes the game look identical on every device for the first time — **CHOOSE THE FACE EARLY**, because every layout is currently sized against a font that was never going to ship.

6 · Very hard, but doable

OPEN-ENDED · BUDGET GENEROUSLY · DO NOT TRANSCRIBE

Nothing here is blocked. Each is expensive because the effort is hard to bound in advance — either it is a taste problem wearing an engineering costume, or it means rebuilding code whose *shape* was set by a browser problem that no longer exists.

Visual parity at 1:1

The look is emergent: 75 keyframe sets, 175 transforms, 110 box-shadows and 68 filters, layered and tuned by eye over many passes. Godot can produce every individual effect. It will not produce the same *accumulation* by transcription, because the compositing model underneath is different.

> **PLAN FOR IT** Expect a first pass around 85% and a deliberate re-tuning pass on top, done against side-by-side captures, with the browser version kept runnable as the reference throughout. Treat it as an art task with an art schedule. The component-scene structure helps here more than anywhere else: re-tuning one `Gauge.tscn` fixes it in three places at once.

Blend modes CSS has and Godot does not `mix-blend-mode: color-dodge · screen`

The logo glow is `color-dodge`; the colour flash and crit wash are `screen`. Godot's `CanvasItem.blend_mode` offers Add, Sub, Mul and premultiplied alpha — `screen` is approximable with Add, `color-dodge` is not.

> **IN GODOT** A shader reading `hint_screen_texture` behind a `BackBufferCopy`, implementing the blend formula directly. Entirely possible; the difficulty is that each backbuffer copy has a real cost and draw ordering must be managed explicitly, on a screen that already stacks several full-screen layers.

The glitch stack `hue-rotate · invert · saturate · contrast · blur, keyframed together`

A six-stop sequence stacking five filter functions at different values per stop, over the whole screen.

> **IN GODOT** One full-screen shader implementing the CSS filter functions — their colour matrices are specified and can be copied exactly — driven by an `AnimationPlayer`. Very achievable; recovering the *feel* of something short, violent and load-bearing is iteration, not implementation.

Un-shaping the code that was shaped around the DOM `view · fx · screens — ~1,300 lines`

A large amount of structure exists only to survive browser behaviour: walking `offsetParent` because transformed elements lie about their rect; suppressing transitions to avoid a visible jump when the line resets; the whitelist-not-blacklist opacity rule for the intro; the mask handoff behind `.intro.masking`; rebuilding panels as markup-only so gesture handlers cannot stack.

> **THE CATCH** None of these problems exist in Godot — which is good news, but it means the code cannot be translated. It has to be re-derived from what it was *for*. This is the largest genuine rewrite, and where behaviour is most likely to be silently lost, because the reasons live in comments rather than tests.
>
> **MITIGATION** Before touching it, write down the observable behaviour of each sequence — what the player sees, in order, with timings — and port against that list rather than against the code.

Keeping the spreadsheet loop as fast as it is now

Today: edit the workbook, double-click `REBUILD.command`, refresh. Sub-ten-second turnaround, and the reason a non-programmer can balance this game.

> **IN GODOT** Achievable but it must be designed. Exporting into `res://` needs a reimport, and a restart in an exported build. The fix is to read the tables from a directory outside the pack at runtime and add a *reload data* key — which gets back to sub-ten-second and arguably better, since it can reload without restarting the fight. It is a system to build, not something that comes free, and skipping it quietly degrades the loop the whole design depends on.

Distribution, if the browser was the point

The current build opens from a URL on a phone, or straight off the disk, with no install. A Godot web export is a 15–40 MB WASM download with real caveats on iOS Safari.

> **THE HONEST FRAMING** A change to the distribution model more than a technical obstacle. If native iOS/Android is the destination, Godot is strictly better and this is a non-issue. If "send a link and they play in ten seconds" is worth keeping, settle it *before* the port starts — it changes what the target is.

7 · Impossible to port

NO TARGET EXISTS · REPLACE, OR DELETE

Nothing here blocks the port. These cannot cross the boundary at all — mostly because they are browser platform, not game. Two are genuine losses; the rest are constraints that vanish and take code with them.

CSS itself, as a design system

There is no cascade in Godot. `game.css` is explicitly ordered so later rules win, sectioned tokens → frame → HUD → stage → lanes → panel → effects → screens. Godot's `Theme` is a flat lookup by control type and variation: no inheritance, no specificity, no `var()` arithmetic.

> **REPLACEMENT** A `Theme` resource for the token layer — the 208 custom properties map well onto theme colours and constants — plus explicit code for what the cascade did implicitly. The *discipline* the stylesheet header describes (one shape per role; to make something stand out change its background, never its geometry) must be rewritten as a Theme convention, or it will be lost. The component-scene structure in section 3 is what replaces it in practice.

The edit-stylesheet-and-refresh loop

The tightest feedback loop in the project, and the one that made this visual identity possible.

> **NEAREST THING** Live editing while the game runs from the editor, plus exported variables, `@tool` scripts and hot-reloading shaders. Shaders in particular recover much of it — editing a `.gdshader` while the game runs is very close to editing CSS. Good, not identical. This is a real cost of the move.

`file://` and everything built to survive it

"Classic scripts, not ES modules, so this opens by double-clicking `index.html`" is a stated ground rule, and a surprising amount of code exists only to honour it: `data.js` compiled because a browser cannot read a CSV from disk; the non-gapless `<audio>` fallback for when `fetch` is blocked; the `postMessage` bridge because every `file://` document has an opaque origin; the autoplay-delegation workaround; every file sharing one global scope with load order as the dependency graph.

> **OUTCOME** All deleted, not ported — there is nothing on the other side for it to become. Net: several hundred lines of workaround removed, one convenience lost.

Browser-platform APIs

`navigator.wakeLock`, `matchMedia("(pointer:coarse)")`, `navigator.clipboard` with its iOS fallback, the dev server's case-sensitivity guard, the mtime cache-buster.

> **MAPPING** Some have direct equivalents (`DisplayServer.screen_set_keep_on`, `DisplayServer.is_touchscreen_available`, `DisplayServer.clipboard_set`). The rest — including the case-sensitivity trap that once cost a deploy — are problems that stop existing. Delete them and the comments explaining them.

Automatic text layout

The browser wraps, reflows and shrink-to-fits for free, and several screens rely on it — the how-to-play list, blurbs, station labels, dialogue bubbles.

> **REPLACEMENT** Explicit sizing and `autowrap_mode`, plus — for the Catalan station names especially — a decision about what happens when a label does not fit. The map already has that policy (*names place themselves, and a name that would collide is dropped*); apply it more widely.

The Playwright screenshot loop `.playwright-cli/`

Driving the running game from outside, reading the accessibility tree and the console.

> **REPLACEMENT** Headless Godot with a test scene and in-engine screenshot capture. Different tooling, similar capability — and the simulation half becomes *easier* to test, because it can run with no renderer at all.

8 · What gets better

TODAY	IN GODOT
Changing how an enemy looks means editing JavaScript presentation smuggled into the units sheet as `fx`, `scale`, `bg_bright` strings the code branches on	**Open its scene.** An enemy's appearance, animation and flourishes are nodes it owns. A new enemy is an inherited scene plus a spreadsheet row, and no code changes.
The same thing drawn three times the MS/EC bar exists in battle, the ride HUD and the profile card	**One scene, instanced.** Fix or restyle it once and every appearance follows — automatic propagation is the structural property the whole plan is built to get.
Two apps, iframe-bridged postMessage across an opaque origin, autoplay delegated, a stubbed bridge so neither app has to run the other	**One project, two folders.** `startEncounter()` is a function call; the §13 contract survives as a data shape. Every autoplay and user-activation problem in the map's notes stops existing.
Art trapped in code ten icons and every station symbol as 8×8 string grids in `util.js`	**PNGs in `art/`.** An artist replaces a file. Resolution by filename also kills the whole class of bug where a sheet asks for a symbol no lookup table has.
Effects as filter stacks in a stylesheet	**Shaders with named uniforms** — hot-reloading, GPU-run, sliders in the Inspector, and capable of things CSS cannot do.
Set pieces as interleaved `await` and class toggles the boss's second wind and fall are ~280 lines of imperative sequencing	**`AnimationPlayer` timelines you can scrub.** The shake-under-the-sink that currently needs two separate mechanisms because "one keyframe list doing both would have to spell out every jitter at every depth" is simply two tracks.
One CSS property, two owners the boss's 1.5× scale silently did nothing for three passes: `.sprwrap` had a static `transform` *and* a running `hover` animation, and the animation replaces rather than composes	**The bug cannot be written.** Scale, position and rotation are separate properties on a node, and an animation track names exactly what it drives. This class of "the value was right, the property was already spoken for" failure disappears.
Hiding the interface by class blacklist `.hush` and the intro's `:not(.intro):not(.flash):not(.stage)…`, where a newly added screen is hidden by default and nothing warns	**Node visibility and CanvasLayers.** Hushing for the second wind becomes hiding one layer — and a new element added to the fight is not silently invisible because somebody forgot to name it in a selector.
One 12 fps clock, plus CSS animations manually quantised to match	**Genuinely one clock**, plus `AnimationPlayer` timelines you can scrub instead of 75 hand-tuned keyframe sets in a text file.
Hand-rasterising to avoid antialiasing because `imageSmoothingEnabled` governs scaling only	**Nearest-neighbour is engine-level.** A low-res SubViewport gives crisp pixels for free — so the map can become real nodes, get faster, and become editable.
15,000 lines in one global scope	Real classes and real refactoring. Renaming stops being grep-and-pray.
Audio graph assembled in code with a documented bug where a hidden gain silently cut the music 9 dB	Buses in the mixer, levels visible, the crusher a built-in effect. That class of bug becomes hard to write.
localStorage, quota-limited and browser-clearable	Real save files. No quota, no silent loss.
Touch only	Gamepad and keyboard free via the input map, native mobile builds, haptics, proper app lifecycle.

THE HONEST COUNTERFACTUAL

If the only goal were a native mobile build, wrapping the existing app (Capacitor, Tauri) would get there in days and preserve the CSS workflow entirely. The case for Godot is not packaging — it is that the presentation layer is approaching what CSS can comfortably carry, that a second app now shares state with the first, and above all that **the project needs to become something an artist can open**. That last one is the argument, and CSS cannot give it to you.

9 · Suggested order of work

Do not port file by file. Port in layers, starting with the layer that has no presentation at all — it is the only part that can be proved correct before anything is drawn.

0 Settle four questions

BEFORE ANY CODE

Does the spreadsheet stay the source of truth? (Assume yes.) Is a browser build still required? Which font ships? And is the scene architecture in section 3 agreed? Each changes the target; each is cheap now and expensive later.

1 Data pipeline and simulation, headless

SMALLEST, HIGHEST VALUE

Retarget the Python tools to emit `res://data/`; write the `DataTable` autoload; port `model`, `kinds`, `rules`, `hooks`, `battle`. Build a harness that replays scripted turns and asserts on the ledger — the project has no such test today and this is the moment to get one. Nothing is drawn.

2 Export the art, choose the font

UNBLOCKS EVERY ARTIST

Run the existing glyph generators once to write `art/icons/`, `art/emotions/`, station rings, arrows and charge nodes as PNGs. Pick and import the font. Cut the 9-patch panel set. After this, nothing visual is trapped in a string table.

3 Solve the cross-cutting three, then build the components

THE ARCHITECTURAL SPINE

`glow.gdshader`, the `.pxr` 9-patch StyleBox, and `ramp.gdshader` — they appear on nearly every element, and deciding them once is what stops the presentation port becoming a thousand small decisions. Then build `components/`: Gauge, Station, tags, tooltip, floating number. Everything after this instances them.

4 Battle: shell, then `_Enemy.tscn`, then the twelve

THE BULK OF THE HOURS

Gauges and rings first as shaders (they prove the pixel pipeline). Then the lane, the ability panel, effects, screens and dialogue. Build the enemy base scene properly before making any inherited enemy — everything it lacks will be lacked twelve times. Port *against a written list of observable behaviour*, not against `view.js`.

4b The set pieces, last of the battle work

MOST VISIBLE, MOST RETUNABLE

`_Boss.tscn`, then `SecondWind.tscn`, `BossFall.tscn` and `EscapeRing.tscn` as timelines. Deliberately after the ordinary fight works: these are the sequences that interrupt everything else, and they are much easier to build against a fight that already resolves correctly than to debug alongside one that does not.

5 Map: nodes, not a rasteriser

LARGEST FILE COUNT, ONE REAL DECISION

Build the SubViewport, instance `StationMarker.tscn` per row, links as textured `Line2D`. Then routing, world state, the vault, the six ride scenes, markers and the trip preview. The systems code is mostly logic and moves easily.

6 Connect them, and delete the bridge

ONE AFTERNOON

The seam is already designed and already stubbed. In Godot it is a scene change and a Dictionary.

7 Audio

DEFERRABLE, PARALLELISABLE

Buses first, then pre-rendered SFX from the sheet, then the theme and per-enemy swaps. Nothing depends on it.

8 **Parity pass** — OPEN-ENDED — RESERVE REAL TIME

Side by side with the browser build, screen by screen. This is where the last 15% is recovered, and it is a taste exercise. Keep the web version runnable until it is signed off.

10 · Things to be careful about

- **The comments are the specification.** This codebase records *why* in prose — the shape-fill ceiling that stops rings merging into a filled triangle; the whitelist-not-blacklist opacity rule; the weighted draw that broke when the weights summed past 1.0; the `SCHEMA.list` change made for one app that silently killed 28 lines of dialogue in the other. Almost none of it is in tests. Extract these rules into a checklist before rewriting the code that carries them.
- **Do not port the workarounds.** Much of `view.js` / `fx.js` is shaped by browser behaviour with no Godot equivalent. Translating it faithfully imports complexity to solve problems that no longer exist. Identify each and delete it deliberately.
- **One number, one place.** The single failure mode of the architecture in section 3 is a balance value that ends up in both the sheet and a `.tscn`. Scenes hold presentation; the sheet holds rules; the read-only Inspector mirror exists so nobody is tempted to type the number twice.
- **Build the base scenes before the instances.** Anything missing from `_Enemy.tscn` is missing twelve times, and retrofitting it into scenes that have since been hand-edited is exactly the mess inherited scenes exist to prevent.
- **Keep the two-app wall standing.** The directory-level separation exists for stated reasons: avoiding cross-contamination, hallucinated design and runaway scope. It becomes `battle/` and `map/` under one project with a shared data autoload — do not dissolve it just because the engine makes dissolving it easy.
- **Protect the designer loop.** If the spreadsheet round trip gets slower, balancing slows down, and balancing is where this game is actually made. Build the runtime data reload early rather than treating it as polish.
- **The set pieces have rulings, not just timings.** The gate that intercepts the killing blow, the charge she keeps, the recharge that runs off the display clock, the escape that refuses during a second wind — these read as implementation detail and are design. They are listed in 3.10; treat that list as a test plan.
- **Do not author game rules that are not yet designed.** The GDDs are written by hand and deliberately incomplete in places — the combat doc's ability list was once hallucinated by an agent and had to be struck through, and `layer_types` and `synergies` are deliberately empty sheets holding a shape for a design not yet written. Where a rule is missing, ask; do not invent.
- **§16 of the combat GDD is a list of traps this project has already fallen into** — fourteen of them, each with the symptom that made it hard to see. Read it before writing the presentation layer, not after.